Министерство науки и высшего образования Российской Федерации

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

ITMO University

ЛАБОРАТОРНАЯ 6

По дисциплине Виртуализация сетевых функций

Тема работы Цепочка сервисных функций (SFC) и жизненный цикл VNF под управлением SDN

Обучающийся Титов Георгий Константинович

Факультет Прикладной информатики

Группа К3422

Направление подготовки 11.03.02 Инфокоммуникационные технологии и системы связи

Образовательная программа Программирование в инфокоммуникационных системах

Обучающийся _____ _____ Титов Г. К.

(дата) (подпись) (Ф.И.О.)

Руководитель _____ _____ Аминов Н.С.

(дата) (подпись) (Ф.И.О.)

Санкт-Петербург 2026

СОДЕРЖАНИЕ

Стр.

1 Цель работы и персональные параметры 3

2 Персональные параметры и программная среда 4

3 Архитектура и схема NFVI 6

3.1 Соответствие учебной реализации ETSI NFV MANO 7

4 Ход выполнения работы 8

4.1 Этап 1. Развёртывание NFVI и управление OVS 8

4.2 Этап 2. Дескрипторы и конфигурация VNF 10

4.3 Этап 3. Построение и проверка SFC 12

4.4 Этап 4. Scale-out — создание FW2 14

4.5 Этап 5. Heal — восстановление после отказа FW1 15

4.6 Этап 6. Производительность NFVI 18

4.7 Этап 7. Финальная проверка и фиксация результатов 19

5 Итоговая конфигурация и история изменений 21

5.1 Конечный VNFFG 21

5.2 Дескрипторы VNF 21

6 Вывод 23

1 Цель работы и персональные параметры

Цель работы – построить программно управляемую цепочку сетевых функций на базе NFV и SDN, проверить фильтрацию и ограничение скорости передачи данных, продемонстрировать масштабирование и восстановление сервиса после отказа VNF.

В ходе работы требовалось развернуть NFVI с Open vSwitch и OpenDaylight, настроить дескрипторы VNF, создать VNFFG, применить правила OpenFlow, провести функциональные тесты, выполнить scale-out и heal, оценить производительность и зафиксировать конечное состояние.

2 Персональные параметры и программная среда

Параметр	Значение
STUDENT	titov
N	87
Управляющая сеть	10.106.87.0/24
OpenDaylight	10.106.87.100
Open vSwitch	10.106.87.10
Client 1	10.87.6.11/24
Client 2	10.87.6.12/24
Server	10.87.6.100/24
Блокируемый TCP-порт	8000 + 87 = 8087
Скорость Shaper	10 + (87 mod 10) = 17 Мбит/с
Цепочки	titov-c1; titov-c2

В качестве NFVI использовались контейнеры Docker, объединённые в топологию Containerlab. Коммутатор реализован Open vSwitch с userspace datapath (netdev), SDN-контроллер – OpenDaylight с интерфейсом RESTCONF и протоколом OpenFlow 1.3. Сетевая функция FW использует nftables, функция ограничения скорости – Linux tc с дисциплиной очереди TBF.

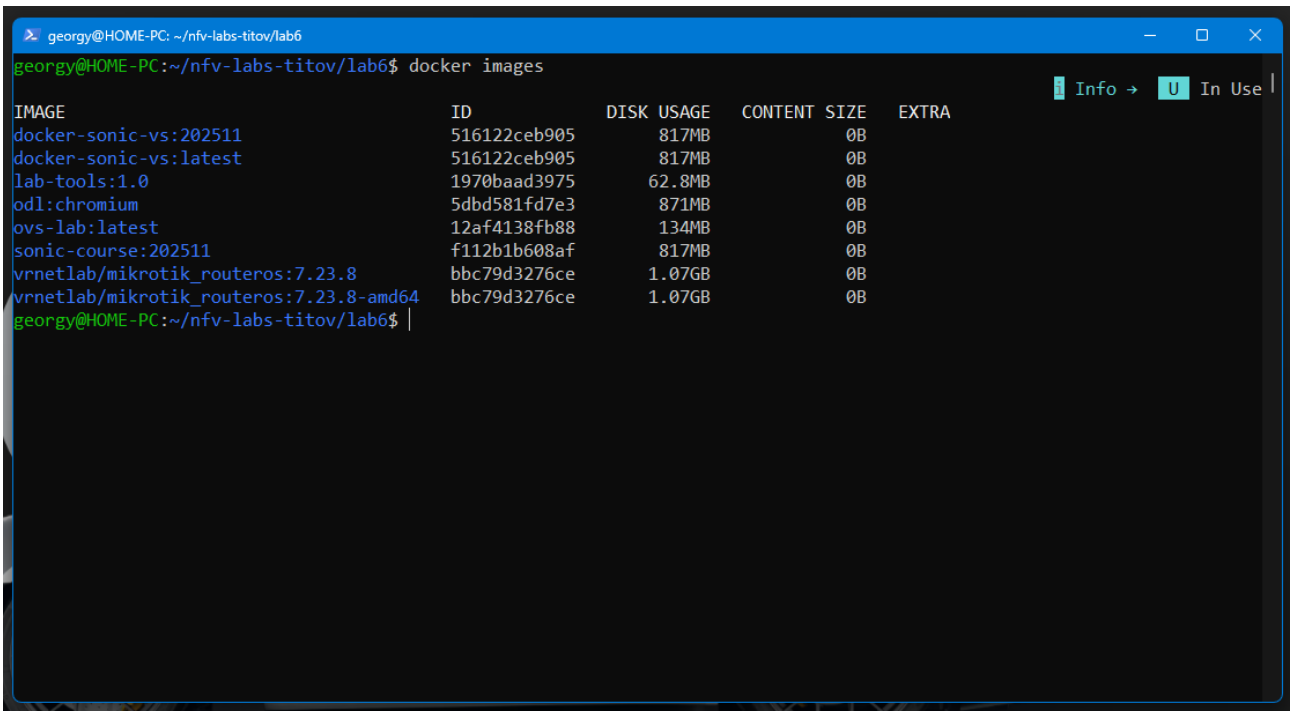


Рисунок 2.1 — Контейнерные образы, подготовленные для лабораторной среды

3 Архитектура и схема NFVI

Логический путь прямого трафика: Client 1 / Client 2 → OVS → Firewall → OVS → Shaper → OVS → Server. Возврат пакетов из каждой VNF в коммутатор нужен для применения следующего OpenFlow-правила. В обратном направлении цепочка проходит через те же функции в обратном порядке.

OpenDaylight соединён с OVS по управляющей сети (TCP/6653). Скрипт nfvo.py передаёт описание потоков контроллеру через RESTCONF (HTTP/8181); пользовательские пакеты контроллером не пересылаются.

Порт OVS	Устройство	Назначение
1	client1:eth1	Трафик первого клиента
2	client2:eth1	Трафик второго клиента
3 / 4	fw1:eth1 / eth2	Исходный межсетевой экран
5 / 6	fw2:eth1 / eth2	Масштабированный межсетевой экран
7 / 8	shaper1:eth1 / eth2	Ограничение скорости
9	server:eth1	Сервер HTTP и iperf3

Firewall и Shaper включены по схеме bump-in-the-wire. Внутри каждого контейнера интерфейсы eth1 и eth2 объединены Linux-мостом br0; VNF не требуют отдельных IP-адресов в подсети 10.87.6.0/24. Мост br0 в контейнере sw – отдельный мост Open vSwitch.

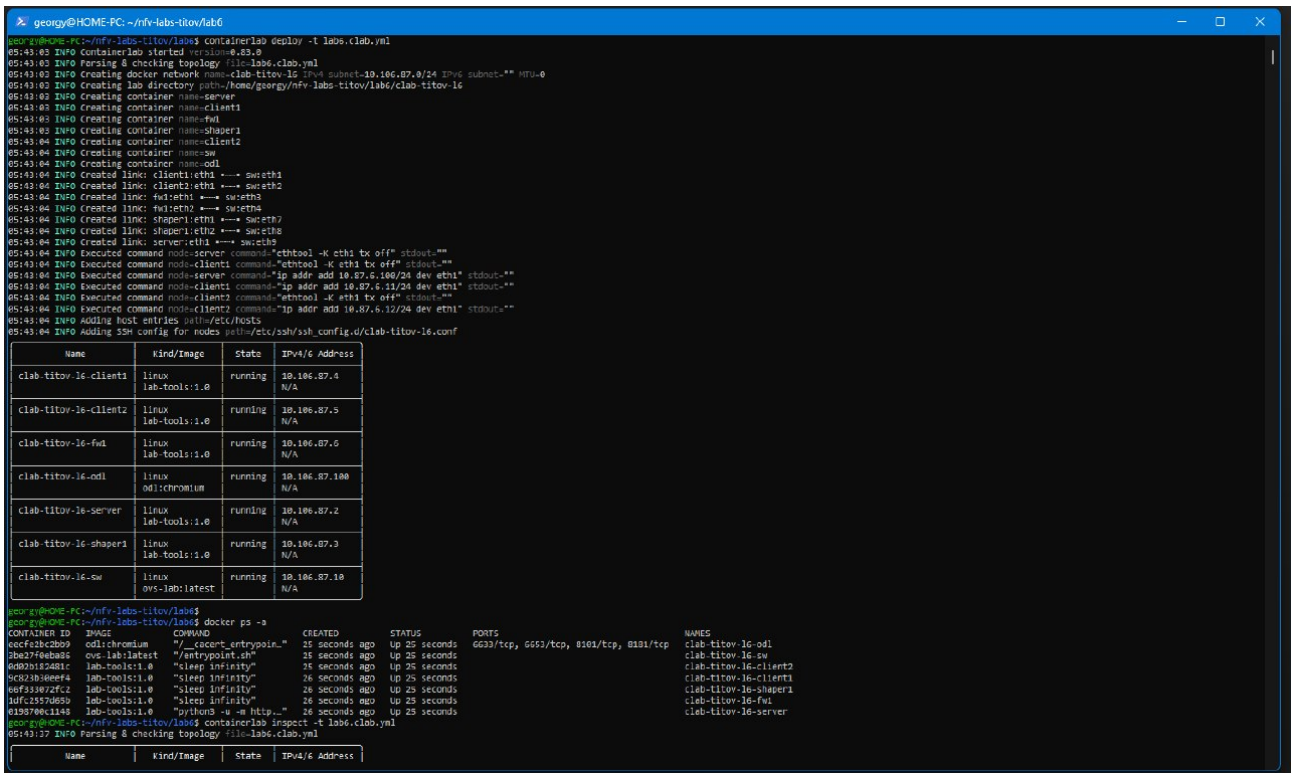


Рисунок 3.1 — Развёртывание топологии Containerlab и проверка подключения OVS к OpenDaylight

3.1 Соответствие учебной реализации ETSI NFV MANO

Элемент архитектуры	Реализация
NFVI	Docker, Linux, сетевые интерфейсы veth, OVS
VIM (упрощённый)	Docker + Containerlab
SDN-контроллер / WIM	OpenDaylight
VNFM (упрощённый)	Bash-скрипт vnfm.sh
NFVO (упрощённый)	Python-скрипт nfvo.py
VNFD	descriptors/fw-vnfd.yaml, descriptors/shaper-vnfd.yaml
VNFFGD (упрощённый)	vnffg.yaml

Приведённое соответствие демонстрирует функциональные роли компонентов; учебные скрипты не реализуют полный набор стандартных интерфейсов и функций ETSI NFV MANO.

4 Ход выполнения работы

4.1 Этап 1. Развёртывание NFVI и управление OVS

Сначала были установлены параметры STUDENT=titov и N=87. По файлу lab6.clab.yml с помощью Containerlab развёрнуты семь исходных узлов: client1, client2, server, sw, odl, fw1, shaper1. На сервере дополнительно запущены iperf3 (5201/TCP) и HTTP-служба (8087/TCP), помимо HTTP на порту 80.

```
export STUDENT=titov N=87
containerlab deploy -t lab6.clab.yml
docker exec -d clab-titov-l6-server iperf3 -s
docker exec -d clab-titov-l6-server python3 -m http.server 8087
```

Листинг 4.1: Развёртывание и запуск проверочных сервисов

На OpenDaylight установлены функции odl-openflowplugin-flow-services-rest и odl-openflowplugin-app-table-miss-enforcer. Скрипт scripts/sw-setup.sh настраивал OVS, задавал протокол OpenFlow13, userspace datapath netdev, fail-mode=secure и подключение к контроллеру 10.106.87.100:6653.

```
docker exec -t clab-titov-l6-odl /opt/odl/bin/client \
  "feature:install odl-openflowplugin-flow-services-rest odl-openflowplugin-app-ta
docker exec -i -e ODL_IP=10.106.87.100 clab-titov-l6-sw sh < scripts/sw-setup.sh
docker exec clab-titov-l6-sw ovs-vsctl show
```

Листинг 4.2: Настройка контроллера и коммутатора

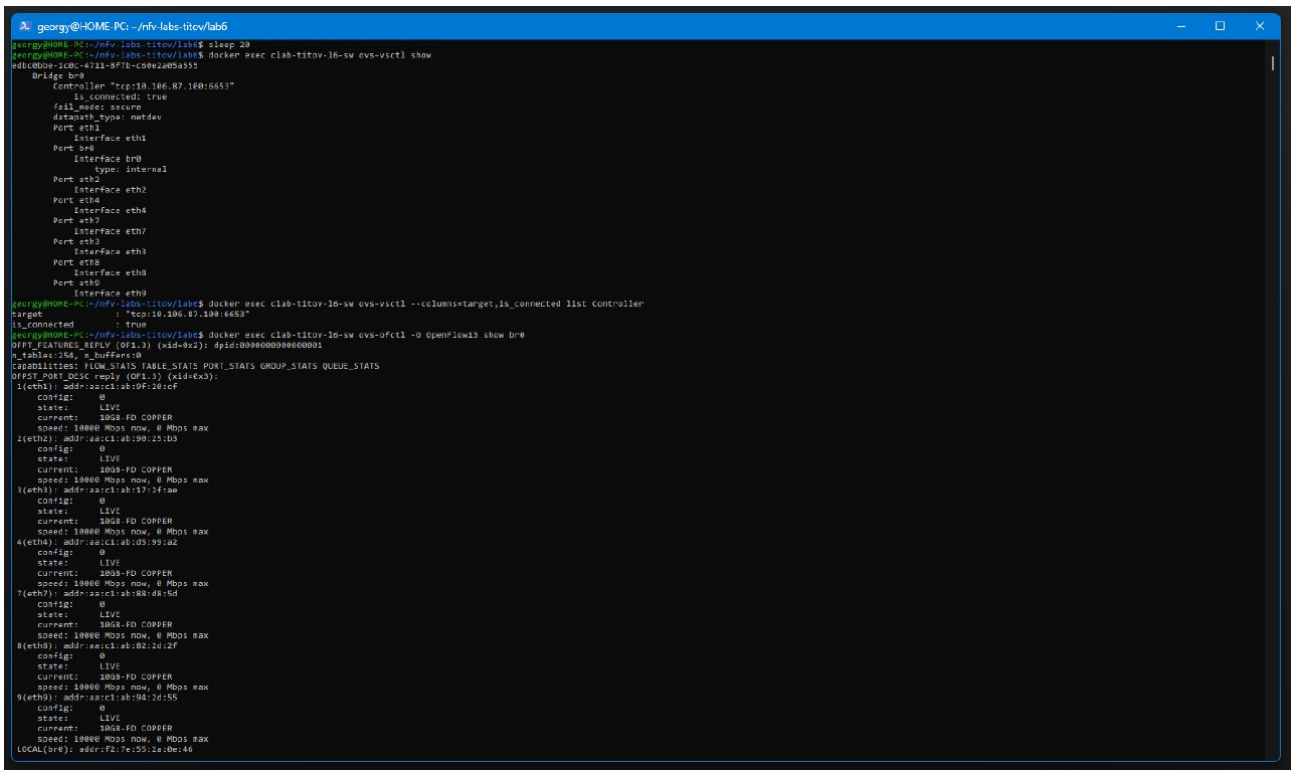


Рисунок 4.1 — Параметры OpenDaylight, OpenFlow и таблицы OVS

До применения VNFFG клиент и сервер, несмотря на нахождение в одной IP-подсети, не имели связи: OVS не располагал правилами пересылки трафика через сервисную цепочку.

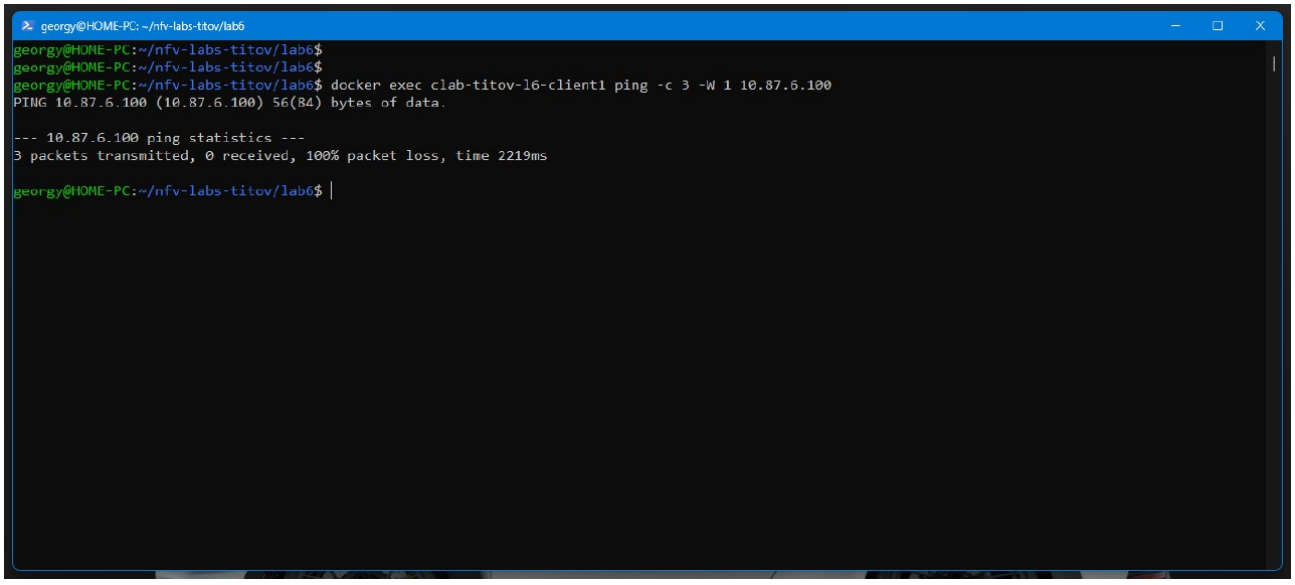


Рисунок 4.2 — Отсутствие связности client1–server до установки сервисной цепочки

4.2 Этап 2. Дескрипторы и конфигурация VNF

Изучен fw-vnfd.yaml и подготовлен дескриптор shaper-vnfd.yaml. Описаны идентификаторы VNF, образ контейнера, VDU, внешние точки подключения eth1/eth2, day-1 конфигурация, масштабирование и операции жизненного цикла. В поле provider для FW указан titov. Дескрипторы имеют учебный упрощённый формат.

```
./vnfm.sh configure fw1 fw
./vnfm.sh configure shaper1 shaper
./vnfm.sh list
docker exec clab-titov-l6-fw1 nft list ruleset
docker exec clab-titov-l6-shaper1 tc qdisc show dev eth2
```

Листинг 4.3: Настройка и проверка сетевых функций

FW1 фильтрует IPv4-трафик при помощи nftables: запрещён TCP/8087, разрешены TCP/80 и ICMP, прочие пакеты учитываются счётчиком и по политике цепочки принимаются. Правила являются stateless. Шейпер применяет TBF к исходящему интерфейсу eth2, ограничивая передачу от клиентов к серверу.

Исходное правило table bridge vnf_fw не поддерживалось используемым ядром WSL2 (ошибка Not supported). Для работы в данной среде была сохранена резервная копия правил и применено семейство table ip vnf_fw с параметром net.bridge.bridge-nf-call-iptables=1. Практическая блокировка пакетов подтверждена ростом счётчика drop.

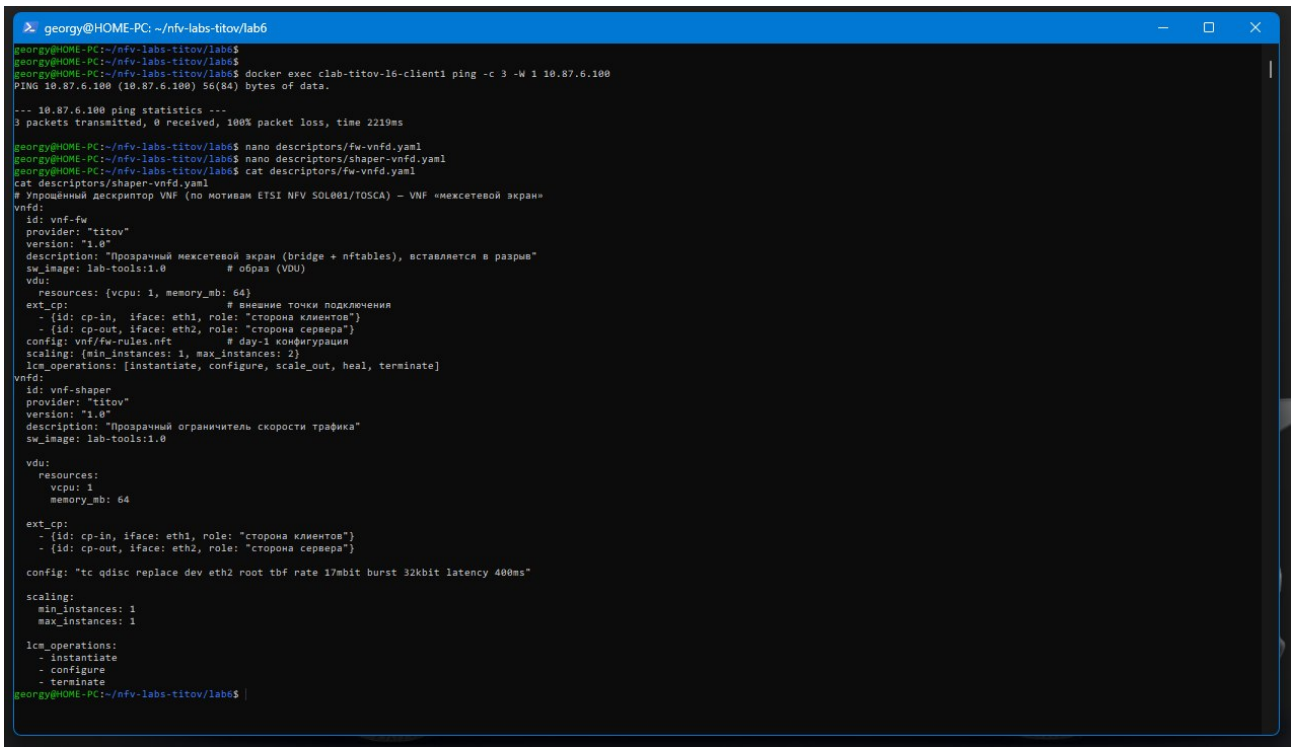


Рисунок 4.3 — Настройка VNF и проверка конфигурации сервисных функций

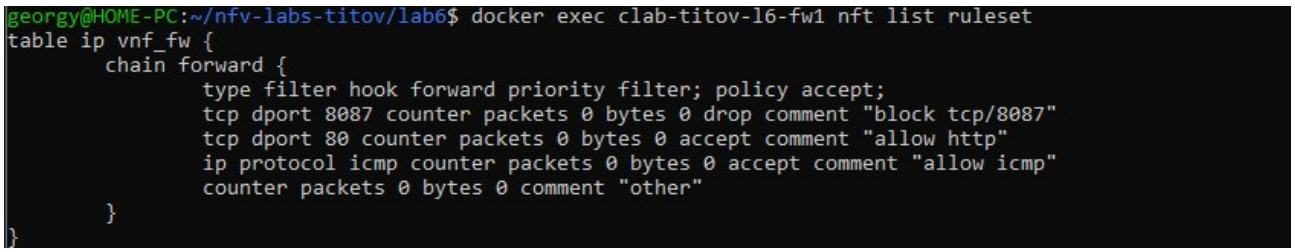


Рисунок 4.4 — Применённые правила nftables для Firewall (таблица семейства ip)

Первоначальный параметр burst 32kbit приводил к потерям пакетов и прерыванию прямого TCP-теста iperf3. Значение увеличено до 256kbit без изменения rate=17mbit; изменение сохранено и в vnfm.sh, и в дескрипторе Shaper.

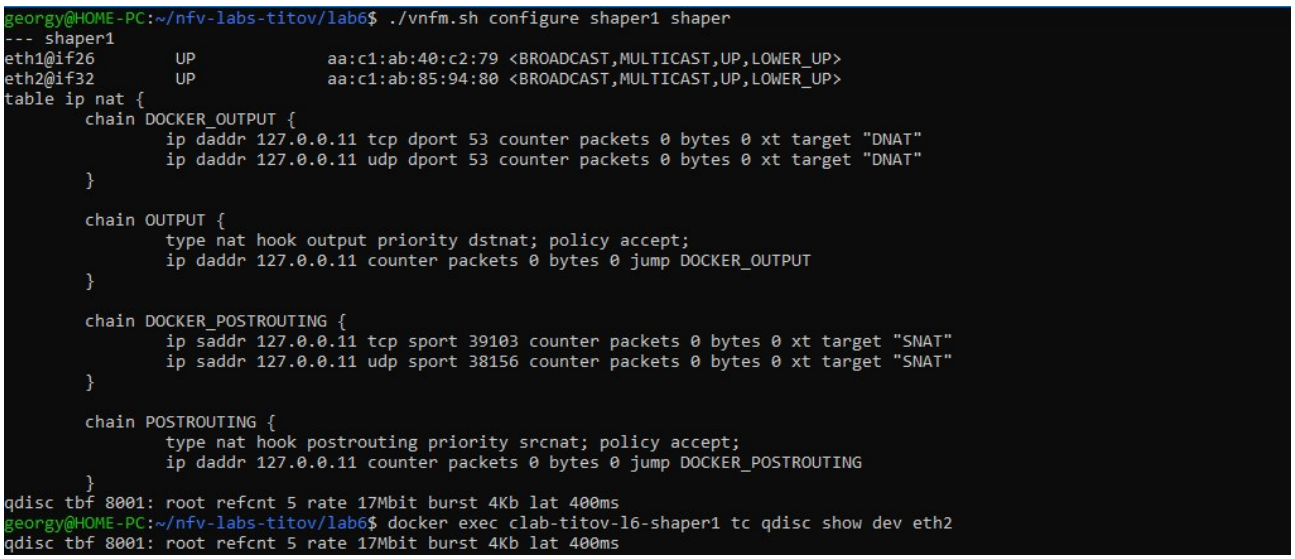


Рисунок 4.5 — Конфигурация Shaper и дисциплины очереди TBF

4.3 Этап 3. Построение и проверка SFC

Файл vnffg.yaml сначала задавал две цепочки: titov-c1 (client1 → fw1 → shaper1 → server) и titov-c2 (client2 → fw1 → shaper1 → server). Скрипт nfvo.py прочитал описание цепочек и установил в OVS через OpenDaylight 24 правила: на каждый из трёх переходов в обоих направлениях создавалось по одному IPv4- и одному ARP-правилу для каждого клиента.

```
python3 nfvo.py apply vnffg.yaml
# applied: 24 flows, removed stale: 0
docker exec clab-titov-l6-sw ovs-ofctl -O OpenFlow13 dump-flows br0
python3 nfvo.py show vnffg.yaml
```

Листинг 4.4: Создание и просмотр OpenFlow-потоков

Направление c1 через FW1	Вход OVS	Выход OVS
Client1 → FW1	1	3
FW1 → Shaper1	4	7
Shaper1 → Server	8	9
Server → Shaper1	9	8
Shaper1 → FW1	7	4
FW1 → Client1	3	1

Правила для прямого IPv4-трафика сопоставляют адрес источника 10.87.6.11; обратные правила – адрес назначения 10.87.6.11. Для ARP используются соответствующие поля sender protocol address и target protocol address. Это позволяет обработать как IPv4-пакеты, так и определение MAC-адресов.

```
georgy@HOME-PC:~/nfv-labs-titov/lab6$ docker exec clab-titov-l6-client1 ping -c 3 -W 2 10.87.6.100
PING 10.87.6.100 (10.87.6.100) 56(84) bytes of data.
64 bytes from 10.87.6.100: icmp_seq=1 ttl=64 time=2.49 ms
64 bytes from 10.87.6.100: icmp_seq=2 ttl=64 time=1.14 ms
64 bytes from 10.87.6.100: icmp_seq=3 ttl=64 time=1.19 ms

--- 10.87.6.100 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2185ms
rtt min/avg/max/mdev = 1.139/1.607/2.490/0.624 ms
georgy@HOME-PC:~/nfv-labs-titov/lab6$ docker exec clab-titov-l6-client2 ping -c 3 -W 2 10.87.6.100
PING 10.87.6.100 (10.87.6.100) 56(84) bytes of data.
64 bytes from 10.87.6.100: icmp_seq=1 ttl=64 time=2.26 ms
64 bytes from 10.87.6.100: icmp_seq=2 ttl=64 time=1.27 ms
64 bytes from 10.87.6.100: icmp_seq=3 ttl=64 time=1.34 ms

--- 10.87.6.100 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2184ms
rtt min/avg/max/mdev = 1.270/1.624/2.262/0.451 ms
georgy@HOME-PC:~/nfv-labs-titov/lab6$
```

Рисунок 4.6 — Проверка доступности сервера с обоих клиентов после применения VNFFG

Проверка TCP/80 вернула HTTP 200. Обращение к TCP/8087 не получило HTTP-ответа (curl: 000), при этом счётчик правила drop увеличился. Следовательно, блокировка подтверждена именно на Firewall, а не только по ошибке клиента.

```
georgy@HOME-PC:~/nfv-labs-titov/lab6$ docker exec clab-titov-l6-client1 curl -sS -m 5 -o /dev/null -w 'HTTP %{http_code}\n' http://10.87.6.100/
HTTP 200
georgy@HOME-PC:~/nfv-labs-titov/lab6$ docker exec clab-titov-l6-fw1 nft list ruleset | grep 'block tcp'
                tcp dport 8087 counter packets 0 bytes 0 drop comment "block tcp/8087"
georgy@HOME-PC:~/nfv-labs-titov/lab6$ docker exec clab-titov-l6-client1 curl -s -m 3 -o /dev/null -w 'HTTP %{http_code}\n' http://10.87.6.100:8087/
HTTP 000
georgy@HOME-PC:~/nfv-labs-titov/lab6$ docker exec clab-titov-l6-fw1 nft list ruleset | grep 'block tcp'
                tcp dport 8087 counter packets 3 bytes 180 drop comment "block tcp/8087"
```

Рисунок 4.7 — Проверка разрешённого HTTP и блокировки TCP/8087

После корректировки burst успешный прямой тест iperf3 показал на стороне приёмника 16,9 Мбит/с при ограничении 17 Мбит/с. В статистике tc учитывались события overlimits и потери пакетов; overlimits означает работу ограничителя, а не само по себе потерю пакета.

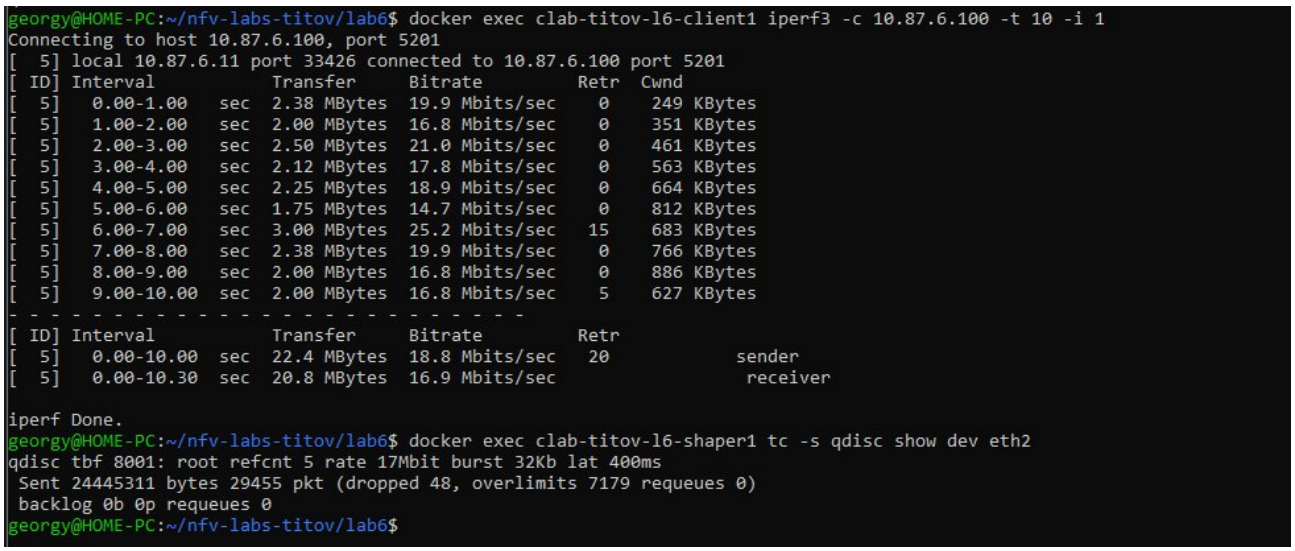


Рисунок 4.8 — Успешный прямой iperf3 после увеличения burst до 256kbit

4.4 Этап 4. Scale-out — создание FW2

Скрипт vnfm.sh создал новый контейнер fw2 из lab-tools:1.0, присоединил eth1 и eth2 через пары veth к портам 5/6 OVS, включил их в мост OVS и применил nftables. После этого только цепочка c2 была изменена на path: [fw2, shaper1]. Повторное применение VNFFG сохранило 24 потока, но изменило входные и выходные порты цепочки c2.

```
./vnfm.sh instantiate fw2 fw 5 6
#  vnffg.yaml: c1 -> [fw1, shaper1], c2 -> [fw2, shaper1]
python3 nfvo.py apply vnffg.yaml
```

Листинг 4.5: Горизонтальное масштабирование межсетевого экрана

Оба клиента сохранили связность. Три HTTP-запроса с client2 увеличили счётчик tcp dport 80 на FW2 с 0 до 18 пакетов; показатель FW1 остался равным 6. Этим подтверждено перенаправление конкретной клиентской цепочки на новый экземпляр.

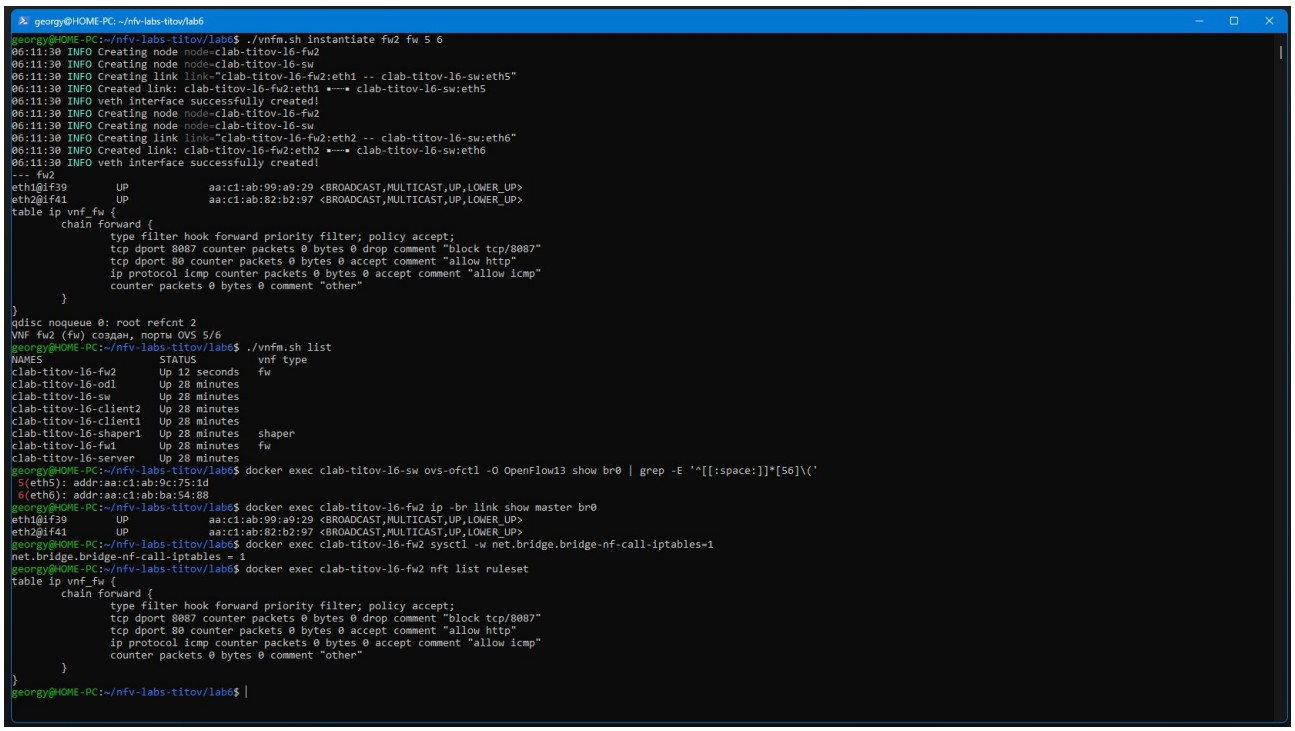


Рисунок 4.9 — Создание FW2, подключение к OVS и изменение цепочки c2

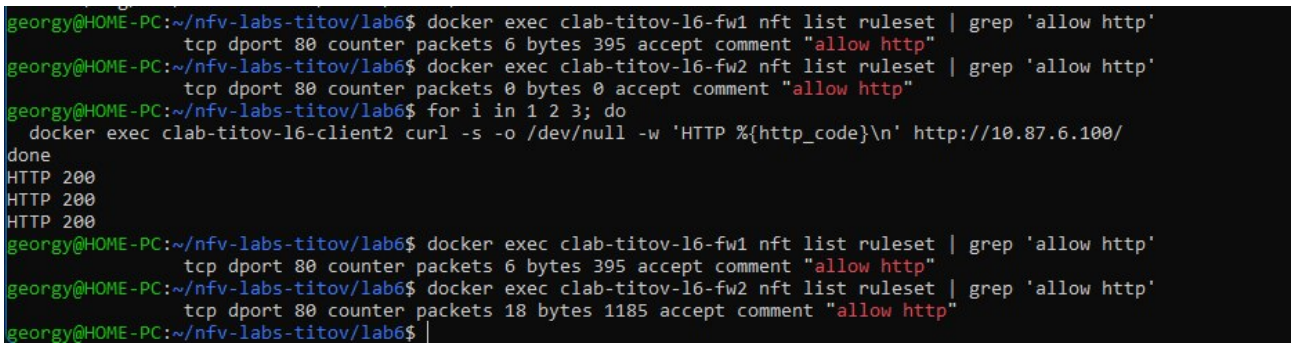


Рисунок 4.10 — Сравнение счётчиков HTTP на FW1 и FW2 после переноса client2

4.5 Этап 5. Heal — восстановление после отказа FW1

Для моделирования отказа выполнена остановка контейнера fw1. Client1, использовавший FW1, потерял связь с сервером (100% потерь). Client2, уже перенесённый на FW2, сохранил связь (0% потерь).

```
docker stop clab-titov-l6-fw1
docker exec clab-titov-l6-client1 ping -c 3 -W 1 10.87.6.100
docker exec clab-titov-l6-client2 ping -c 3 -W 1 10.87.6.100
```

Листинг 4.6: Моделирование отказа и диагностика

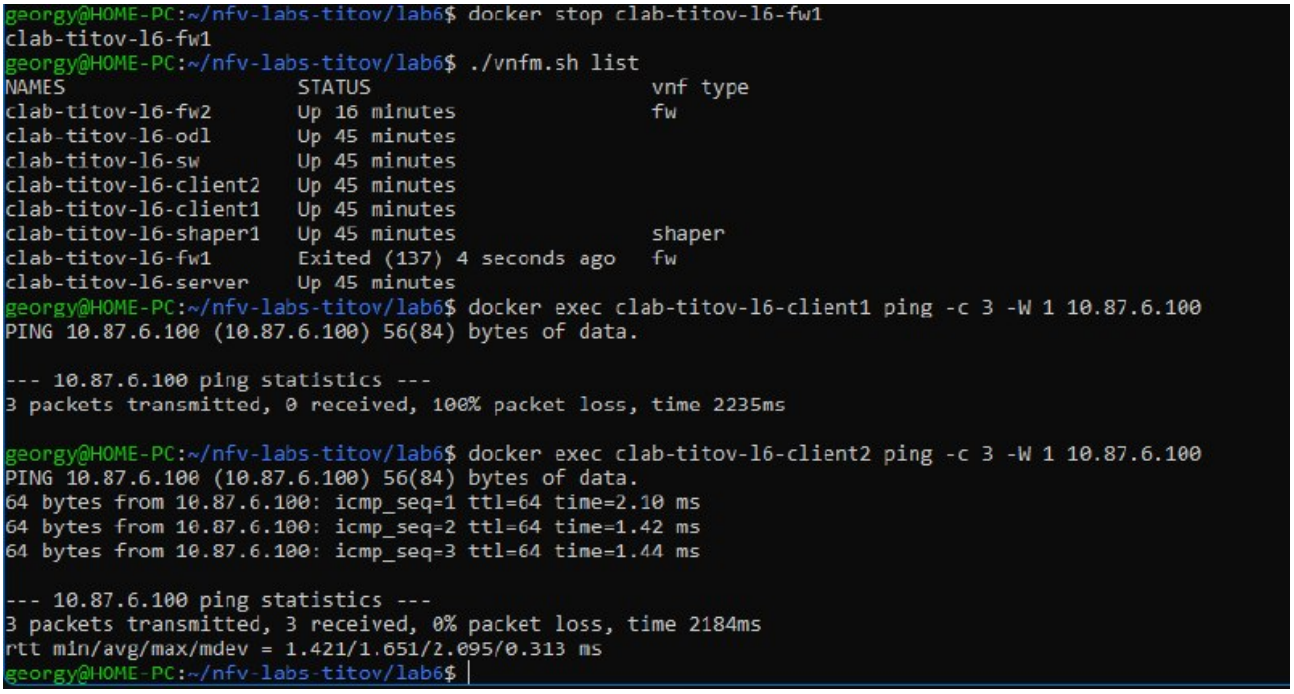


Рисунок 4.11 — Отказ FW1: потеря связи client1 при сохранении связи client2

В vnffg.yaml обе цепочки переведены на [fw2, shaper1], после чего nfvo.py обновил потоки. Ping и HTTP с client1 вновь заработали. Прямые IPv4-переходы c1 стали 1→5, 6→7 и 8→9; обратные – 9→8, 7→6 и 5→1. Зафиксированная длительность ручного сценария от отметки начала до проверки восстановления – 110 с. Это не время автоматического переключения и включает действия оператора.

```
#   VNFFG:
chains:
  - {name: "${STUDENT}-c1", from: client1, to: server, path: [fw2, shaper1]}
  - {name: "${STUDENT}-c2", from: client2, to: server, path: [fw2, shaper1]}
python3 nfvo.py apply vnffg.yaml
./vnfm.sh terminate fw1 3 4
```

Листинг 4.7: Восстановление сервисных цепочек и завершение FW1

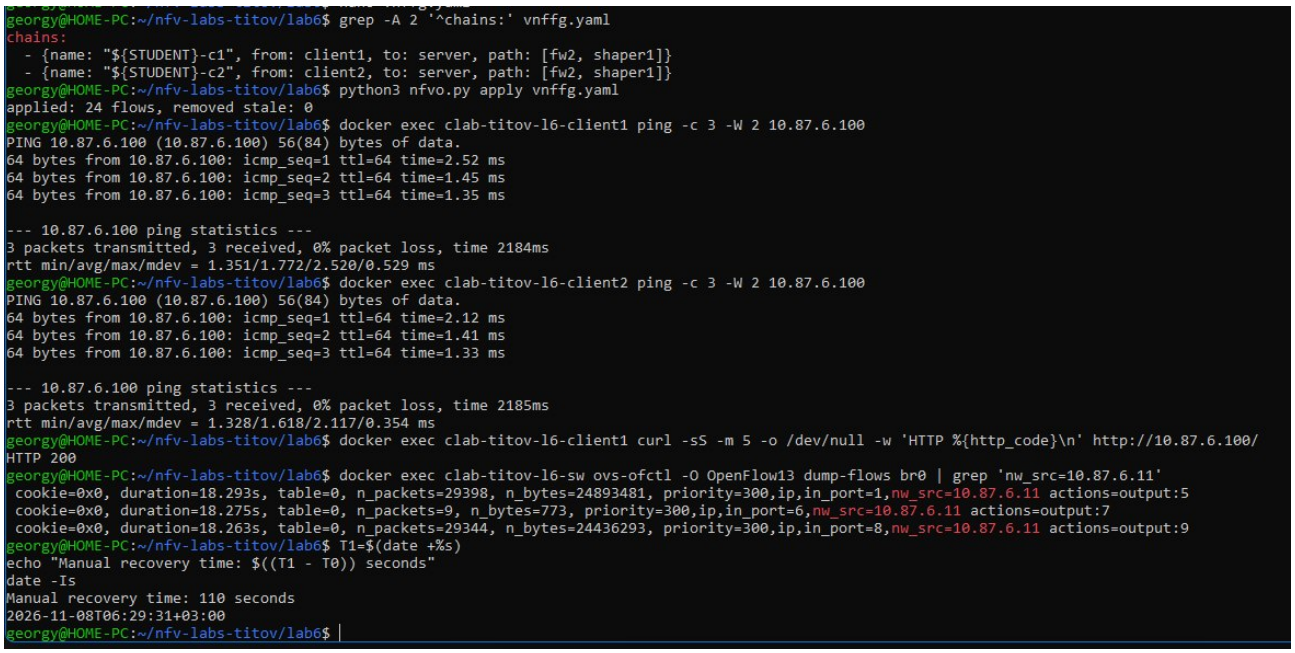


Рисунок 4.12 — Восстановление client1 после Heal и измерение ручного времени 110 с

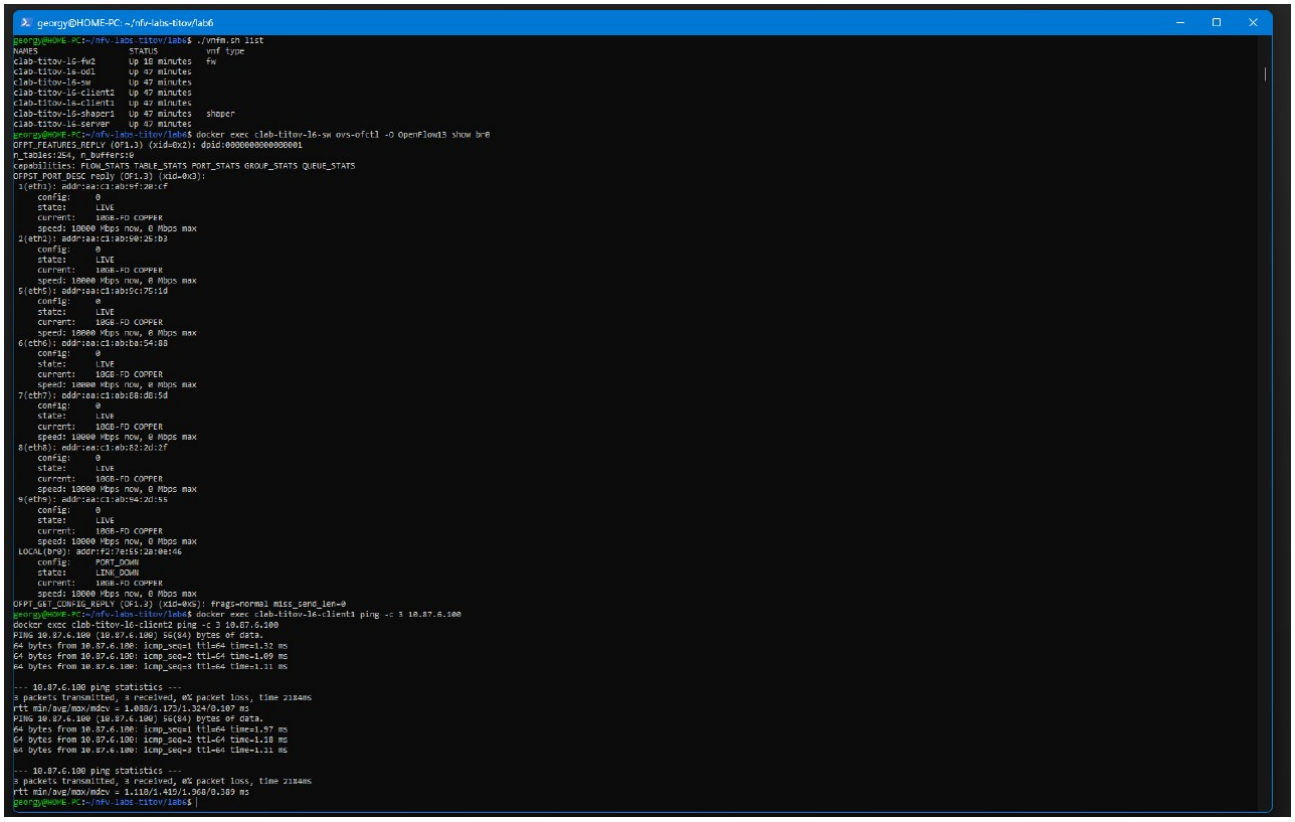


Рисунок 4.13 — Состояние после удаления FW1: FW2 и Shaper1 остаются активными

После terminate контейнер FW1 и порты 3/4 удалены, при этом оба клиента продолжили обмениваться ICMP-пакетами с сервером без потерь.

4.6 Этап 6. Производительность NFVI

После Heal выполнено измерение iperf3 в обратном направлении с четырьмя параллельными TCP-потоками (режим -R -P 4). Средняя суммарная пропускная способность приёма составила 1,54 Гбит/с за 10 секунд, с 9264 TCP retransmissions на передающей стороне. В прямом направлении скорость ограничивалась TBF до 16,9 Мбит/с.

```
docker exec clab-titov-l6-client1 iperf3 -c 10.87.6.100 -t 10 -R -P 4
docker exec clab-titov-l6-sw sh -c "top -b -n 1 | head -15"
```

Листинг 4.8: Оценка производительности

```
[ ID] Interval           Transfer     Bitrate         Retr
[  5]   0.00-10.00  sec   444 MBytes   372 Mbits/sec  2155             sender
[  5]   0.00-10.00  sec   442 MBytes   371 Mbits/sec                   receiver
[  7]   0.00-10.00  sec   406 MBytes   340 Mbits/sec  2126             sender
[  7]   0.00-10.00  sec   405 MBytes   340 Mbits/sec                   receiver
[  9]   0.00-10.00  sec   584 MBytes   490 Mbits/sec  3114             sender
[  9]   0.00-10.00  sec   583 MBytes   489 Mbits/sec                   receiver
[ 11]   0.00-10.00  sec   404 MBytes   339 Mbits/sec  1869             sender
[ 11]   0.00-10.00  sec   403 MBytes   338 Mbits/sec                   receiver
[SUM]   0.00-10.00  sec  1.79 GBytes  1.54 Gbits/sec  9264             sender
[SUM]   0.00-10.00  sec  1.79 GBytes  1.54 Gbits/sec                   receiver

iperf Done.
georgy@HOME-PC:~/nfv-labs-titov/lab6$ docker exec clab-titov-l6-sw sh -c "top -b -n 1 | head -15"
top - 03:34:44 up  1:13,  0 users,  load average: 0.29, 0.10, 0.03
Tasks:   6 total,   1 running,   5 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :   7944.6 total,   6120.9 free,   1589.0 used,    384.9 buff/cache
MiB Swap:   4096.0 total,   4096.0 free,      0.0 used.   6355.6 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
      1 root      20   0    2588   1732   1636 S   0.0   0.0   0:00.01 sleep
      9 root      20   0   18396   9888   7436 S   0.0   0.1   0:00.26 ovsdb-s+
     12 root      20   0  397436  21088   8412 S   0.0   0.3   0:27.40 ovs-vsw+
    155 root      20   0    2680   1924   1808 S   0.0   0.0   0:00.00 sh
    161 root      20   0    7452   5020   2972 R   0.0   0.1   0:00.00 top
    162 root      20   0    2596   1596   1492 S   0.0   0.0   0:00.00 head
georgy@HOME-PC:~/nfv-labs-titov/lab6$
```

Рисунок 4.14 — Итоги iperf3 (-R -P 4) и состояние процессов OVS

В момент запуска top система показывала CPU idle 100%, но команда top выполнялась после завершения iperf3. Поэтому вывод не позволяет

оценить пиковую загрузку CPU под нагрузкой. Потенциальные накладные расходы обусловлены OVS userspace datapath (AF_PACKET) и переходами пакетов через сетевой стек и мосты VNF. Для ускорения применяют DPDK, OVS-DPDK, SR-IOV, vhost-user или SmartNIC; они требуют дополнительных ресурсов и могут уменьшать гибкость виртуализации.

4.7 Этап 7. Финальная проверка и фиксация результатов

Финальная команда вывела дату и имя хоста, список работающих экземпляров, счётчики OpenFlow и nftables, ответ HTTP 200 и результат iperf3. В окончательном состоянии FW1 отсутствует, FW2 и Shaper1 активны. Для FW2 подтверждено увеличение счётчика блокировки tcp dport 8087 с 0 до 3 пакетов после запроса client1, а прямой iperf3 повторно показал 16,9 Мбит/с.

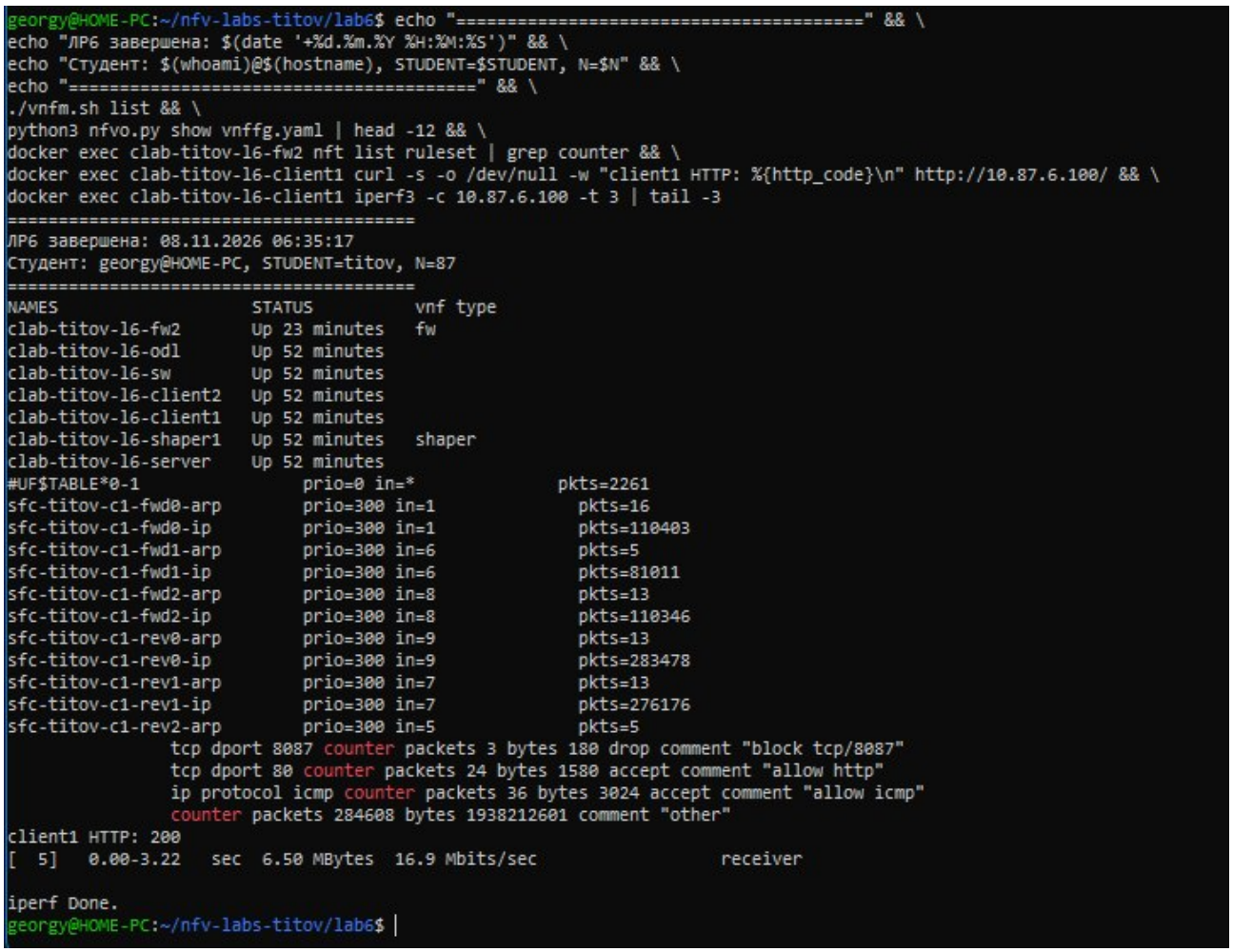


Рисунок 4.15 — Финальный артефакт: состав VNF, OpenFlow, nftables, HTTP и iperf3

5 Итоговая конфигурация и история изменений

5.1 Конечный VNFFG

```
controller: {url: "http://10.106.${N}.100:8181", user: admin, password: admin}
switch: "openflow:1"
endpoints:
  client1: {port: 1, ip: "10.${N}.6.11"}
  client2: {port: 2, ip: "10.${N}.6.12"}
  server:  {port: 9, ip: "10.${N}.6.100"}
vnfs:
  fw1:     {type: fw, in: 3, out: 4}
  fw2:     {type: fw, in: 5, out: 6}
  shaper1: {type: shaper, in: 7, out: 8}
chains:
  - {name: "${STUDENT}-c1", from: client1, to: server, path: [fw2, shaper1]}
  - {name: "${STUDENT}-c2", from: client2, to: server, path: [fw2, shaper1]}
```

Листинг 5.1: Итоговый vnffg.yaml (переменные N и STUDENT подставляются средой)

5.2 Дескрипторы VNF

```
vnfd:
  id: vnf-fw
  provider: "titov"
  version: "1.0"
  description: "   (bridge + nftables)"
  sw_image: lab-tools:1.0
  vdu:
    resources: {vcpu: 1, memory_mb: 64}
  ext_cp:
    - {id: cp-in, iface: eth1, role: " "}
    - {id: cp-out, iface: eth2, role: " "}
  config: vnf/fw-rules.nft
  scaling: {min_instances: 1, max_instances: 2}
  lcm_operations: [instantiate, configure, scale_out, heal, terminate]
```

Листинг 5.2: FW VNFD; описание приведено по исходному дескриптору с подтверждённой заменой provider

```
vnfd:
  id: vnf-shaper
  provider: "titov"
  version: "1.0"
  description: "    (tc tbf)"
  sw_image: lab-tools:1.0
  vdu:
    resources: {vcpu: 1, memory_mb: 64}
  ext_cp:
    - {id: cp-in, iface: eth1, role: " "}
    - {id: cp-out, iface: eth2, role: " "}
  config: "tc qdisc replace dev eth2 root tbf rate 17mbit burst 256kbit latency 40
  scaling: {min_instances: 1, max_instances: 1}
  lcm_operations: [instantiate, configure, terminate]
```

Листинг 5.3: Shaper VNFD, реконструкция по заданию и подтверждённой конфигурации; точный полный файл пользователя не предоставлен

Git зафиксировал два основных изменения: d2c28fd «lab6: chain c2 -> fw2» и 57254cc «lab6: heal c1 -> fw2». Первый коммит был создан после scale-out, поэтому первоначальное состояние обеих цепочек через FW1 отдельно в истории не сохранено.

6 Вывод

В лабораторной работе развёрнута управляемая NFVI с прозрачными контейнерными сетевыми функциями Firewall и Shaper. Через OpenDaylight и OpenFlow реализованы две сервисные цепочки, подтверждены IPv4/ARP-пересылка, фильтрация порта TCP/8087 и ограничение прямого трафика до 16,9 Мбит/с. Создан FW2, выполнено распределение клиентских цепочек между экземплярами и ручное восстановление после отказа FW1 с последующим удалением старого экземпляра. Итоговое состояние успешно проверено. Дополнительно подготовлен start-lab6.sh для повторного развёртывания конечной конфигурации после перезапуска среды; этот скрипт не реализует автоматический мониторинг и heal.